

Java Generics

Reusable, Safe, and Modern Code

CODEHIVE

CODEHIVE

1. The Power of Generics

Generics (introduced in Java 5) allow you to write classes, interfaces, and methods that work with different data types without specifying the exact type in advance.

Why use Generics?

- **Code Reusability:** Prevent creating "StringBox", "IntBox", and "DoubleBox" separately.
- **Type Safety:** Catch incompatible types at **Compile Time** instead of crashing at Runtime.
- **Cleaner Code:** Eliminates the need for messy manual Type Casting (e.g., `(String) list.get(0)`).

2. Creating a Generic Class

In a generic class, we use a placeholder (usually **T**) to represent the type that will be provided later.

```
class Box<T> {  
    T value;  
  
    void set(T value) { this.value = value; }  
    T get() { return value; }  
}
```

Implementation:

```
Box<String> sBox = new Box<>();  
sBox.set("Software");  
  
Box<Integer> nBox = new Box<>();  
nBox.set(101);
```

3. Type Parameter Naming

While you can use any letter, Java developers follow these standard conventions:

- **T** for Type
- **E** for Element (used in Collections)
- **K** for Key
- **V** for Value
- **N** for Number

4. Generic Methods

You can make a single method generic, even if the class it belongs to is not generic. The type parameter is placed before the return type.

```
public static <T> void printArray(T[] array) {  
    for (T item : array) {  
        System.out.println(item);  
    }  
}
```

Java's **Type Inference** automatically determines the type based on what you pass in (String[] vs Integer[]).

5. Bounded Types

Sometimes you want to use Generics but restrict the allowed types. For example, a math class should only accept **Numbers**.

```
// T must be a subclass of Number
class Stats<T extends Number> {
    T[] nums;

    Stats(T[] nums) { this.nums = nums; }

    double average() {
        double sum = 0;
        for (T n : nums) sum += n.doubleValue();
        return sum / nums.length;
    }
}
```

6. Generics with Collections

The most common application of Generics is in the Java Collections Framework (ArrayList, HashMap). Before Generics, everything was stored as Object, which required constant casting.

The Modern Way:

```
ArrayList<String> list = new ArrayList<>();
```

The compiler now ensures only Strings are added, and you get a String back immediately upon calling `get()`.

7. Wildcards (?)

Wildcards represent an unknown type. They are useful in method parameters where you want flexibility.

- `<?>`: Unbounded wildcard (any type).
- `<? extends T>`: Upper bounded (T or its subclasses).
- `<? super T>`: Lower bounded (T or its superclasses).

8. Type Erasure

A curious fact: At runtime, Java "forgets" the generic types. This is called **Type Erasure**.

The compiler replaces all generic types with their bounds (or `Object`) and inserts the necessary casts. This ensures compatibility with older versions of Java (pre-Java 5).

9. Summary & Final Tips

- Generics allow for type-agnostic but **type-safe** code.
- The diamond operator `<>` allows the compiler to infer types during variable initialization.
- Use `extends` to define upper bounds.
- Generics cannot be used with primitives—use **Wrapper Classes** instead!

© 2026 CodeHive Academy | Advanced Java Series